Formal and computational companion to Paper V Preprint 1.0

Juan Pablo Traverso Gianini

October 18, 2026

This companion describes the final formal development and the computational material accompanying PAPER_V_preprint_v1.0_en.md and PAPER_V_preprint_v1.0_es.md. Paper V concerns general finite packing and physical realization. Paper IV proves the all-order bound up to an absolute additive constant and the sharp eventual maximum discussed in the main paper. The separate IV–V application does not supply a premise of V's general results.

1. Canonical source and public interface

The canonical entry point is `PaperV.lean`. Its public import closure contains 122 local modules. File inclusion, compilation, public import and theorem availability are checked separately: the aggregate build reaches the proof modules, the statement gates emit exact types and axiom footprints, and the full audit visits the declarations of that same closure. The source package also contains standalone gates; they are validation programs, not additional assumptions of the public theorems.

The environment is `leanprover/lean4:v4.35.0`, with Mathlib pinned to 56a005855ea17e0b0958f73ec801ae140b0a1679. The source archive is PAPER_V_preprint_v1.0_FORMAL_SOURCE_435.zip; its exact digest and per-file manifest are recorded in FREEZE_VALIDATION.json. The source headers credit Juan Pablo Traverso Gianini for the formalization and integration while retaining the mathematical attributions and source/port credits already present. They do not reattribute classical results.

Check	Final scope
Public root	`PaperV`
Aggregate build	Exit 0; 2582 jobs
Full declaration audit	Exit 0; 2838 declarations, 122 modules, 5547 visited
Exact type and axiom gates	10 gates; 403 type queries and 431 axiom queries
Nonstandard axioms	Zero
Foundational axiom union	`propext`, `Classical.choice`, `Quot.sound`
Main and supporting-interface correspondences	37 and 11

The source archive contains the checked local Lean sources, package configuration, license texts and a relative per-file manifest. The separate 02_validation/PREPRINT_1_0/ directory contains the final logs, exit receipts and exact public closure. Before visiting declarations, the audit checks that every listed module is present in the imported environment. Mathematical attributions and source credits are retained in the canonical development.

2. Architecture and hypotheses

The conflict-system and bounded-copy modules define feasible sets, colorability defects and transfer inequalities. The resource-certificate modules connect actual capacity relaxations and primal–dual certificates to occurrence multisets. Realizer and host modules turn a selected color class into one physical packing; the argument does not take a union of independently feasible classes. The copy, pure-profile and mixed-promotion modules control shared physical resources. Unit-boundary modules provide restriction, completion and dual lifting. Finite sensitivity and finite-tree accounting have their own explicit scope.

Several hypotheses deserve to remain visible beside the declaration map. The multiplicity parameter satisfies $q \geq 1$. A weighted target and the number of classes are fixed before selecting a certificate. The finite bounded-resource LP uses nonempty resource sets where required; the general resource relaxation permits empty resource sets. Admissibility families used to take a minimum are nonempty. A physical realizer must satisfy the actual disjoint-resource conditions. Rational inputs and rational certificates are related to real optima by proved bridges. Copy and mixed-copy conclusions retain their multiplicity and shared-resource assumptions. Generic sensitivity results do not identify an unproved WSNF family; finite-tree bounds are not global asymptotic loss bounds.

2.1. Coverage of the supporting results

Item	Checked result and boundary
F1	Actual finite resource LP, rational primal/dual certificates, physical class realizations and the two-error bound, including rational-to-real optimum bridges
F2	Minimum-cost finite certificate portfolio for a fixed target and number of classes; compatible physical unions with disjoint resources
F3	Degree-sum greedy host assignments, physical resource soundness, inherited rational weights and avoidance of a fixed background packing
F4	Weighted paired-factor averaging and physical completion of the survivors, including the empty-base case
F5	Exact unit-boundary restriction/extension, explicit dual lifting, integral and fractional optimum formulas and gap conservation; original gains nonnegative, reduced central gains possibly signed
F6	Physical mixed triangle/K4 promotion, capacity preservation, value identities and a bounded normalization sequence preserving rationality
F7	Generic finite-polytope sensitivity, continuity, dense-family extension and finite-gap stability; no unproved WSNF identification or classification is asserted
F8	The checked finite-tree accounting retains its fractional slack and exact weights; it is not a global asymptotic loss estimate

3. SEAL: implementation, guarantees and finite certificates

The mathematical results remain in Section 8 of the main paper. Proposition 8.1 establishes feasibility of the specified operations, Proposition 8.2 bounds accepted improving repairs, and Theorem 8.3 gives the stated exact or guaranteed host regimes. Their formal declarations appear in Section 5 below. Lean verifies these mathematical interfaces under their hypotheses; it does not verify the Python, JavaScript or C++ implementation end to end, or its polynomial runtime.

3.1. Current implementation and models

The software is distributed in 03_reproducibility/SEAL/. Its `current/` directory contains the Python BASE routines, the v3 driver and the JavaScript/C++ BASE ports. `replay/` contains

complete test inputs, saved outputs, exact checkers and current result tables. The software manifest fixes the exact source bytes described here.

The BASE variants use standard cardinality, nonnegative pair weights, degree capacities, or a tracked potential as specified by the input model. The v3 driver is restricted to unit weights and capacity one. Unsupported weighted or higher-capacity v3 calls are rejected; the corresponding BASE variants remain separate interfaces. The ports expose physical selected families as well as objective values, permitting independent feasibility checks in every tested language.

The current N1 operation is transactional. It tentatively releases a blocking assignment, checks relocation to another compatible host, then checks insertion of the uncovered pair into the original host. It commits only a feasible move with strictly positive objective gain. Failure restores the complete prior state. Thus the applicable objective and capacity constraints must be supplied before interpreting a run.

```
R := a feasible host assignment
for an uncovered pair e and a compatible host j:
    choose a blocking assignment (f, j)
    save the current state and release (f, j)
    try relocating f to another compatible host
    try inserting e into j
    commit only if feasible and the objective strictly increases
    otherwise restore the saved state
```

The BASE and v3 fallback routines implement N1. The separate `reference_v1/` routine implements N1 and N2 for the unweighted, capacity-one model; it is not an implementation of all BASE variants. In that reference, scarcity selection is followed by exhaustive finite neighborhood search, restarting after an accepted improvement. An optional LP audit runs after construction.

The v3 bipartite branch validates the proper list assignment it obtains before using it. The executable is not claimed to construct an assignment throughout the full BKW theorem's range. A successful individual assignment or matching lower and upper bounds supplies its own finite certificate. No general approximation guarantee for local search follows from the exact mathematical special regimes.

3.2. Current finite results

Current-suite check	Verified result
Python physical packings	110: 72 BASE and 38 v3
JavaScript and C++ physical packings	54 in each language; all objective values agree with Python
Complete rational cardinality duals	38; 149,819 object inequalities and 47,015 rows checked
Cardinality optima certified by equal bounds	12: six release cases, two witnesses and four larger cases
Positive certified intervals	26: twelve release gaps of 1–3 and fourteen larger-case gaps of 1–40
Independent negative controls	Seven rejected

The BASE tests use eighteen saved inputs in each of four modes. Weighted inputs use weights in `{1,2,5,10}`; the higher-capacity inputs assign capacity two to every host. Each port checks standard, weighted and capacity-two modes on those inputs. The v3 tests use unit weights and capacity one: eighteen release instances, two bipartite witnesses and eighteen larger saved

configurations. Weighted and capacity-two feasibility and objective checks do not supply optimality certificates for those models. The complete current per-instance values are recorded in replay/CURRENT_RESULTS.csv.

An explicit physical N1 witness improves one selected pair to two. The additional finite transaction regression checks 685 feasible initial states and 27,708 operation trials, with 3,180 accepted insertions and 24,528 rejected trials restoring the state. These counts concern two specified host systems, capacities one/two and gains zero/one; they are not universal correctness claims.

The replay checker reconstructs host containment, global pair uniqueness, per-host degree capacities, cardinalities and weighted objectives from the complete input specifications. Transactional tests check accepted N1 improvements, restoration after failed attempts, zero-weight behavior and the stated capacity models. These are executed finite checks, not universal proofs of program correctness.

For every saved rational dual it reconstructs all objects and constraint rows, checks nonnegative prices and coverage, and recomputes the objective exactly. A feasible packing gives a lower bound; the floor of a valid rational upper bound gives an integral upper bound. Coincident bounds certify that individual output's optimum, while a positive interval records uncertainty. A floating-point solver status or a numerical objective alone is not used as a certificate.

The three-language comparison is limited to the saved input models and outputs. It is not a proof of general equivalence, and runtime measurements are not mathematical premises. Current exact certificates, source hashes and limits are retained in the result report distributed with the software.

The additional capacity-one reference suite contains thirty saved instances and 147,111 host–pair objects. Fourteen intervals certify integral optima; sixteen retain positive uncertainty, of integer width at most 37. Six small optima have independent enumeration. Four altered certificates are rejected and two explicit tests exercise N1 and N2. Twelve compact cases enable repairs; eighteen larger cases run selection and dual auditing only. Odd-set constraints are generated for palettes of size at most nine; omitting larger odd sets can weaken a bound without invalidating a checked dual. This suite does not imply universal exactness of the relaxation.

3.3. Scalar certificate checks

Certo 0.34.0 checks five encoded scalar consequences. They expose arithmetic assumptions and provide replayable checks; they do not prove existence of a graph, physical realizer or feasible packing.

Obligation	Mathematical role	Replay qualification
two_error_transfer	Combine relaxation and realization losses	SMT-dependent
integer_dual_stop	Stop when integer lower and dual upper bounds coincide	SMT-dependent
composed_realization_errors	Add the stated realization error budgets	SMT-dependent
improving_repairs_budget	Bound accepted improving repairs	Solver-free certificate
gap_residual_and_resource_defect	Separate residual and resource defect	SMT-dependent

A sixth, solver-free certificate is a negative control for a deliberately insufficient scalar implication: $\nu = \mathrm{cov} = 0$ and a relaxation value $1/2$ satisfy those encoded premises while leaving positive relaxation gap. It is not a graph counterexample and is not claimed to arise from a

host system. In particular, it does not contradict the resource upper bounds retained by the actual SEAL model. Each replay binds both the specification hash and the obligation; replay against a different obligation is rejected. `RESULTS.json` records which checks require an SMT solver.

4. Reproduction

Extract PAPER_V_preprint_v1.0_FORMAL_SOURCE_435.zip and run from its package root with the pinned dependencies:

```
lake build PaperV
lake env lean PaperVManuscriptGate.lean
lake env lean PaperVSupplementGate.lean
lake env lean PaperVImprovementsGate.lean
lake env lean PaperVReleaseGate.lean
lake env lean FreezeFullAudit.lean
```

The accompanying CHECKED_PUBLICATION_GATES_435.json records all ten gates, exact commands, queried declarations and axiom footprints. MANUSCRIPT_MAP.json and SUPPLEMENT_MAP.json give the exact statement correspondence. SOURCE_CLOSURE .json identifies the public modules reached by the aggregate. No new mathematical axiom is admitted by the audit.

From 03_reproducibility/SEAL/, check the saved outputs with Python's standard library:

```
python replay/scripts/check_outputs.py --source-root current
```

To regenerate the current suite, use the dependencies and runtimes pinned in its README, then run:

```
python replay/scripts/run_suite.py --source-root current
python replay/scripts/run_ports.py --source-root current
python replay/scripts/check_outputs.py --source-root current
```

The additional reference witnesses and scalar certificates have their separate commands:

```
python reference_v1/check_certificate.py
python certo_v/run_certo.py
```

Use normal Python execution, without `-O`, to keep assertions enabled. Rechecking saved witnesses does not require the numerical solver that generated their prices. Generation and Certo's SMT-dependent checks require their documented dependencies; port generation requires Node and a C++17 compiler, with `--cxx` available to specify the compiler. The software manifest binds source, inputs, complete vectors, outputs and validation records.

5. Statement-level declaration map

The following 37 correspondences identify the public declarations. The exact types, assumptions and separate conclusions remain in the source and emitted gate output; this index does not suppress the qualifications in Section 2.

Main-paper item	Canonical declarations
Definition 2.1	PaperV.ConflictSystem; `PaperV.CSFeasible`; `PaperV.nu`
Definition 2.2	`PaperV.Copies`; `PaperV.Hgraph`; PaperV.IsQMultiset
Definition 2.3	`PaperV.Dq`; `PaperV.oct`; PaperV.Dq_one_eq
Definition 2.4	PaperV.Manuscript.ResourceRelaxation; PaperV.ResourceCertificate.System; PaperV.ResourceCertificate.System.incidence

Main-paper item	Canonical declarations
Definition 2.5	PaperV.Manuscript.ResourceRelaxation; PaperV.Manuscript.ResourceRelaxation.Optimum
Lemma 3.1	`PaperV.splitting`
Corollary 3.2	PaperV.nu_ge_ceil
Corollary 3.2a	PaperV.cover_witness_defect; PaperV.triangle_cover_defect
Corollary 3.2b	PaperV.Manuscript.minimum_coloring_rigidity_iff
Theorem 3.3	PaperV.Manuscript.resource_half_integral_transfer
Theorem 3.4	PaperV.Manuscript.resource_integral_transfer
Theorem 3.4w	PaperV.transfer_q_weighted
Lemma 3.5	PaperV.parity_localization
Definition 4.1	PaperV.ClassRealizer; `PaperV.RealCert`; PaperV.RealCert.cost
Theorem 4.2	PaperV.realizable_class_transfer; PaperV.realizable_transfer_cert
Definition 4.3	`PaperV.wRDq`
Theorem 4.4	PaperV.wRDq_identity_eq_wDq; PaperV.wRDq_le_wDq
Example 4.5	PaperV.StarExample.intermediate_defect
Corollary 4.6	PaperV.ResourceCertificate.two_error_real_optimum; PaperV.ResourceCertificate.resource_gap_identity; PaperV.ResourceCertificate.exists_real_optimum
Proposition 5.1	PaperV.Host.csFeasible_hostCS_of_properOn
Theorem 5.2	PaperV.Galvin.galvin_multigraph_max_degree; PaperV.Galvin.galvin_bkw_local
Corollary 5.3	PaperV.Host.galvin_host_completion_exact; PaperV.Host.bkw_host_completion_exact; PaperV.Host.galvin_lifted_transfer; PaperV.Host.galvin_local_lifted_transfer
Example 5.5	PaperV.Strictness.strict_separation; PaperV.Strictness.nu_hostCS_kr
Subsidiary Claim 6.1a	PaperV.copy_integral_edgeLoad_le_two
Theorem 6.1	PaperV.copy_fractional
Theorem 6.2	PaperV.copy_integral_for_optimal_packings; PaperV.copy_integral_for_packings; PaperV.triangleImageMultiset
Lemma 6.2k	PaperV.third_subset_union; PaperV.k4_of_three_triangles; PaperV.triCS_clique_card_le_four
Corollary 6.2a	PaperV.copy_triple_bound; `PaperV.tight_oct`
Proposition 7.1	PaperV.PureProfile.prop43_nu; PaperV.PureProfile.prop43_muMat
Proposition 7.1a	PaperV.PureProfile.nu_ge_frac_muMatDeg; PaperV.PureProfile.nu_ge_two_thirds_muMatDeg; PaperV.PureProfile.nu_ge_muMatDeg_two
Proposition 7.1b	PaperV.PureProfile.muMatDeg_eq_min; PaperV.PureProfile.nu_eq_muMatDeg_of_even; PaperV.PureProfile.nu_eq_frac_muMatDeg_of_odd
Theorem 7.3	PaperV.Mixed.mixed_copy_fractional; PaperV.mixed_copy_integral
Corollary 7.3a	PaperV.Mixed.mixed_copy_integral_of_roundingGap
Proposition 8.1	PaperV.feasible_run; PaperV.SealBridge.sealFeasible_of_properOn
Proposition 8.2	PaperV.accepted_repairs_le_cov; PaperV.accepted_repairs_le_audit
Theorem 8.3	PaperV.SealBridge.seal_galvin_regime; PaperV.SealBridge.seal_bkw_regime; PaperV.SealBridge.seal_konig_guarantee
Proposition 9.3	PaperV.Steiner.exists_quadratic_defect_frac

The Theorem 6.2 interface takes the two specified feasible copied packings and constructs their actual image multiset; its optimal-packing adapter retains those same families. The Theorem 7.3 interface likewise uses the actual mixed-copy data. Older existential convenience wrappers remain available, but are not used as substitutes for these correspondences.

6. Supporting mathematical interfaces

The following eleven correspondences locate the additional supporting results in the delivered source project. Their labels identify the mathematical source inventory; the exact source types control every hypothesis. No separate undelivered supplement is required to locate these interfaces.

Retained item	Canonical declarations
Proposition 3.6	PaperV.Dq_antitone; PaperV.Dq_le_oct; PaperV.Dq_mono_left
Proposition 3.7	PaperV.Dq_eq_zero_iff; PaperV.oct_eq_zero_iff; PaperV.Dq_eq_zero_iff_chromatic; PaperV.Dq_pos_iff_lt_chromatic
Proposition 3.8	PaperV.Dq_ge_card_sub; PaperV.defect_tight_all_max; PaperV.defect_eq_of_survivors; PaperV.Manuscript.minimum_coloring_rigidity_iff
Proposition 3.9	`PaperV.Dq_sum`; `PaperV.oct_sum`
Proposition 3.10	PaperV.Dq_lineGraph_eq_zero_iff; PaperV.Dq_lineGraph_pos_iff; PaperV.Dq_one_lineGraph; PaperV.profile_zero_set_not_values
Corollary 3.11	PaperV.TerminalPair.p4_133_existence_sharp; PaperV.TerminalPair.p4_133_existence_coarse
Theorem 7.4	PaperV.PairedFactors.exists_paired_completion
Proposition 7.5	PaperV.FiniteLP.gap_identity; PaperV.FiniteLP.weak_duality; PaperV.FiniteLP.complementarity_of_equal_value; PaperV.RationalFarkas.farkas; PaperV.RationalFarkas.not_sat_of_cert; PaperV.FiniteLP.exists_equal_primal_dual
Corollary 5.4	PaperV.Host.weighted_host_selection; PaperV.Host.weighted_host_selection_nu; PaperV.Host.konig_weighted_host_selection; PaperV.Host.galvin_heavy_single_host
Lemma C.1	PaperV.UnitBoundary.completion_optimal_unconditional; PaperV.UnitBoundary.unitBoundary_completion_optimal; PaperV.UnitBoundary.exists_optimal_normPack
Lemma C.2	PaperV.UnitBoundary.exists_endpoint_repacking; PaperV.UnitBoundary.endpoint_repacking_card_bound; PaperV.UnitBoundary.endpoint_repacking_nonloss

Lemma C.1 has an unconditional optimum-formula adapter without an inhabited-universe or representatives hypothesis; its normalized-witness form retains explicit representatives. Lemma C.2 supplies an actual representative family and physical packing, including the empty universe; the transport statement retains its normalized-packing data.

7. Scope of the final record

The main manuscript contains one formalization paragraph; this companion carries the source architecture, exact correspondence, computational account and reproduction commands. Formal checking, finite certificate checking, bibliographic priority and external human review are distinct obligations. The separate IV–V application has its own project and validation record. Paper V neither imports that application's global bound nor depends on Paper VI. This companion accompanies Preprint 1.0. Kernel checking and finite certificate verification do not constitute external human peer review or historical-priority certification.

8. Reproducibility of the finite counterexamples

The directory 03_reproducibility/counterexamples/ supplies bk_completion_certificates .py and the complete saved BK certificate. Run `python bk_completion_certificates.py --verify-only` from that directory with assertions enabled. The standard-library checker validates the rational dual of value 19/2, enumerates all 70 optimal nine-element packings and checks the residual parameter beta=2. A changed dual-price negative control is rejected. Certificate search without `--verify-only` optionally uses SciPy; saved-certificate verification does not. The exact finite count 70 is a computational result, not a declaration proved by the Lean kernel. Kernel-checked declarations separately establish the BK optimum and relevant bounds, and the stated CLON-2, STAR-T2 and Steiner facts.

9. Licenses and attribution

Copyright (c) 2026 Juan Pablo Traverso Gianini. Written materials are released under CC BY-NC 4.0. The author's Lean and software contributions are released under Apache 2.0; upstream dependencies and third-party contributions retain their own notices and licenses. The source headers and accompanying license texts make this distinction explicit. Classical mathematical attributions are retained; a formalization credit does not reattribute the underlying theorem.